

Southeast University

Department of Computer Science and Engineering

Assignment

Submitted By :

Name : Tiash Khan Toha

ID : 2024200000134

Batch: 68

Course Code: CSE281.3

Course Title : Introduction to Programming Language II (Java)

Submission Date: 27 September 2026

Submitted To:

Md. Ayon Mia

Department of Computer Science and Engineering

Introduction

This assignment is about building a small Java system that manages the energy of video game characters. When I first read the problem, I thought it would take a lot of time, but once I broke it down into smaller classes, it wasn't that hard.

The main goal is to cover all four OOP principles in a single program and show that we know how to use try-catch, finally, and multi-catch for exception handling.

Below, I explain everything class by class. Then I describe how the OOP principles and exception handling are implemented. At the end, I attach the full code and the output I got when I ran it.

Problem Understanding

After reading the question a few times, this is what I understood: there's an abstract Character class at the top. Then PlayerCharacter extends it, and Mage and Warrior extend PlayerCharacter. It's basically a small family tree, two levels deep.

On top of that, the assignment wants me to actually prove polymorphism works, not just write it and hope it's right. I need to show it by putting different objects in one array and calling the same method on all of them.

Same thing for exceptions: I had to actually make it throw and catch something, not just talk about it.

Class Design

1. Character class

This class is abstract, so it can't be instantiated on its own, only through a child class. It keeps characterID and energyLevel as private fields so nothing outside the class can change them directly.

The constructor is protected on purpose. Only subclasses should call it with super().

calculateRegenRate() is abstract because every character type regenerates energy differently, so it doesn't make sense to implement it here.

displayInfo(), on the other hand, is a normal method that prints the ID and the current energy value.

2. PlayerCharacter class

Extends Character and adds playerName. The constructor calls super() first (required in Java), then clamps negative energy to 0. calculateRegenRate() returns 0.0 since a generic player has no special regen. There are two restoreEnergy() overloads: one takes int, the other double.

3. Mage and Warrior classes

Both extend PlayerCharacter and don't really add any new fields. They just override calculateRegenRate() with their own value: 5% for Mage and 2% for Warrior.

displayInfo() is also overridden in both, mainly to show which class the character belongs to and what the actual regen amount comes out to once you apply the percentage to the current energy.

4. InsufficientEnergyException class

Nothing too complicated here, it just extends the built in Exception class and the constructor takes a message and forwards it to super(). It gets thrown from inside useEnergy() whenever the amount requested is bigger than the current energy of the character.

How the OOP Principles Are Used

Encapsulation is there because all the important fields (characterID, energyLevel, playerName) are private. The only way to access or modify them from outside is through getter/setter methods, never directly.

Inheritance is shown through the class hierarchy: PlayerCharacter extends Character, and both Mage and Warrior extend PlayerCharacter, making it a two-level chain.

For polymorphism, the more interesting part is runtime polymorphism. I stored a Mage object and a Warrior object inside a Character[] array and called displayInfo()/calculateRegenRate() on the array. Even though the declared type of the array is Character, at runtime it still picks the correct overridden version depending on what object is actually in the array.

The compile-time part is the two overloaded restoreEnergy() methods. Java decides which one to call based on whether I pass an int or a double.

Abstraction comes from the Character class being abstract. It doesn't let outside code worry about how the regen rate is calculated; it just forces every subclass to define its own version of calculateRegenRate().

Exception Handling

useEnergy(double amount) checks whether the requested amount is more than the current energyLevel. If it is, it throws an InsufficientEnergyException with a message describing what went wrong.

To test this in main(), I first wrapped a call in a normal try-catch-finally. The finally block always runs, regardless of whether an exception happened, and prints the character's current energy so we can confirm it didn't change after the failed attempt. Then, to also demonstrate a multi-catch, I used catch (InsufficientEnergyException | ArithmeticException e) in another try block. To trigger the ArithmeticException part, I divided a number by zero.

6. Source Code

Character.java

```
public abstract class Character {

    private String characterID;
    private double energyLevel;

    protected Character(String id, double energy) {
        characterID = id;
        energyLevel = energy;
    }

    public abstract double calculateRegenRate();

    public void displayInfo() {
        System.out.println("ID: " + characterID + ", Energy: " + energyLevel);
    }

    protected String getCharacterID() {
        return characterID;
    }

    protected double getEnergyLevel() {
        return energyLevel;
    }

    protected void setEnergyLevel(double energy) {
        energyLevel = energy;
    }
}
```

PlayerCharacter.java

```
public class PlayerCharacter extends Character {public class PlayerCharacter extends
Character {

    private String playerName;

    public PlayerCharacter(String id, double energy, String name) {
        super(id, energy);
        if (energy < 0) {
            setEnergyLevel(0);
        }
        playerName = name;
    }

    public String getPlayerName() {
        return playerName;
    }

    public void setPlayerName(String name) {
        playerName = name;
    }

    public double calculateRegenRate() {
        return 0.0;
    }

    public void displayInfo() {
        super.displayInfo();
        System.out.println("Name: " + playerName);
    }
```

```
 public void restoreEnergy(int amount) {
  setEnergyLevel(getEnergyLevel() + amount);
  System.out.println(playerName + " got " + amount + " energy back, now has " +
 getEnergyLevel());
  }

  public void restoreEnergy(double amount) {
  setEnergyLevel(getEnergyLevel() + amount);
  System.out.println(playerName + " got " + amount + " energy back, now has " +
 getEnergyLevel());
  }

  public void useEnergy(double amount) throws InsufficientEnergyException {
  if (amount > getEnergyLevel()) {
  throw new InsufficientEnergyException(playerName + " does not have enough energy for " +
 amount);
  }
  setEnergyLevel(getEnergyLevel() - amount);
  System.out.println(playerName + " used " + amount + " energy, left with " +
 getEnergyLevel());
  }
 }
```

Mage.java

```
public class Mage extends PlayerCharacter {

    public Mage(String id, double energy, String name) {
        super(id, energy, name);
    }

    public double calculateRegenRate() {
        return 5.0;
    }

    public void displayInfo() {
        super.displayInfo();
        double regen = getEnergyLevel() * calculateRegenRate() / 100;
        System.out.println("Class: Mage, Regen Rate: " + calculateRegenRate() +
"%, Regen Amount: " + regen);
    }
}
```

Warrior.java

```
public class Warrior extends PlayerCharacter {

    public Warrior(String id, double energy, String name) {
        super(id, energy, name);
    }

    public double calculateRegenRate() {
        return 2.0;
    }

    public void displayInfo() {
        super.displayInfo();
        double regen = getEnergyLevel() * calculateRegenRate() / 100;
        System.out.println("Class: Warrior, Regen Rate: " +
calculateRegenRate() + "%, Regen Amount: " + regen);
    }
}
```

InsufficientEnergyException.java

```
public class InsufficientEnergyException extends Exception {

    public InsufficientEnergyException(String msg) {
        super(msg);
    }
}
```

Main.java

```
public class Main {

    public static void main(String[] args) {

        Mage mage = new Mage("C001", 50, "Gandalf");
        Warrior warrior = new Warrior("C002", 80, "Conan");

        Character[] arr = new Character[2];
        arr[0] = mage;
        arr[1] = warrior;

        System.out.println("Showing character info using array (runtime polymorphism)");
        for (Character c : arr) {
            c.displayInfo();
            System.out.println("Regen rate: " + c.calculateRegenRate() + "%");
            System.out.println();
        }

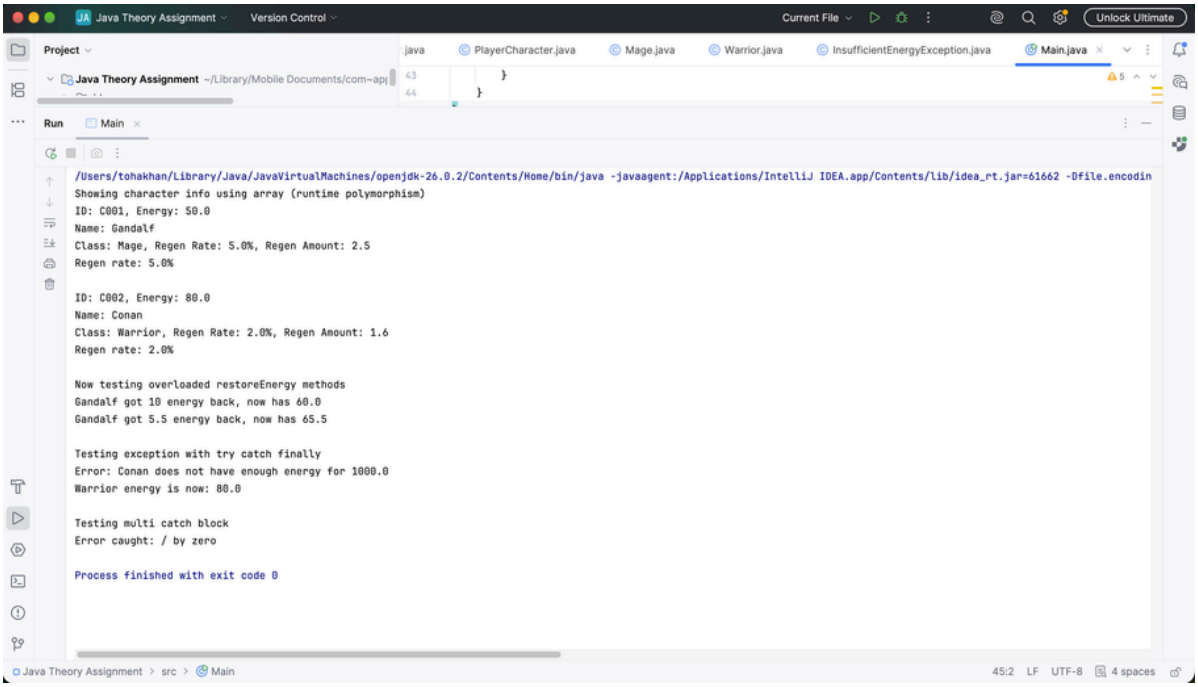
        System.out.println("Now testing overloaded restoreEnergy methods");
        PlayerCharacter p = mage;
        p.restoreEnergy(10);
        p.restoreEnergy(5.5);
        System.out.println();

        System.out.println("Testing exception with try catch finally");
        try {
            warrior.useEnergy(1000);
        } catch (InsufficientEnergyException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Warrior energy is now: " + warrior.getEnergyLevel());
        }
        System.out.println();

        System.out.println("Testing multi catch block");
        try {
            int a = 10;
            int b = 0;
            int result = a / b;
            warrior.useEnergy(result);
        } catch (InsufficientEnergyException | ArithmeticException e) {
            System.out.println("Error caught: " + e.getMessage());
        }
    }
}
```

Program Output

Below is the actual console output produced when running the program:



Conclusion

Overall, I feel like this assignment actually helped. Before this, I understood OOP more from theory/slides, but writing it out made it click better, especially the polymorphism part with the array.

The exception-handling part was also good practice. I hadn't really used multi-catch before, so that was new for me.

If I had more time, I probably would've added a bit more validation (like checking playerName isn't empty). But for what this assignment asked, I think this covers everything needed.